

SENTINEL 2.0

Production Vulnerability & Bug Audit Report

Prepared for: Sentinel Development Team / School Security Deployment Group

Objective: Review of critical bugs, logic vulnerabilities, and performance issues addressed in the Sentinel 2.0 CCTV AI platform to ensure 24/7 continuous system availability.

Issue 1: Database Connection Locking & Deadlocks (LockedConnection)

Severity: **CRITICAL** | **File:** db_schema.py

The Problem: The original LockedConnection class lacked a robust context manager or query timeout mechanism. If any process crashed before calling `.close()`, the global multiprocessing database lock remained locked forever, completely freezing the live surveillance tracking and alerting pipeline.

The Fix & Mitigation: Replaced the fragile locking design with automated database auto-retry watchdogs using exponential backoff (retrying up to 3 times) to handle high write-concurrency. Enabled SQLite Write-Ahead Logging (WAL) and NORMAL synchronization modes to allow simultaneous database readers and writers. Implemented an automatic database integrity check (`integrity_check(1)`) and self-healing system that backs up and regenerates a clean database structure if corruption is detected.

Issue 2: Unguarded Multiprocessing Tracking Buffers (Data Loss)

Severity: **CRITICAL** | **File:** event.py

The Problem: Tracking frames were written to a plain Python list shared across separate camera worker processes. When a process was cycled or terminated, all un-flushed tracking data from that worker was permanently lost, breaking the tracking history and overlay logs.

The Fix & Mitigation: Implemented a reliable process-cycle finalization flow. Wrapped worker processes in finally clean-up blocks in `backend_runner.py` to ensure all tracking events are automatically flushed and sessions are finalized when a worker exits or cycles (after 10,000 frames).

Issue 3: Double Database Connections in Alerts (Lockup Risk)

Severity: **CRITICAL** | **File:** alerts.py

The Problem: Creating an alert and then generating its principal notification opened two separate connections, attempting to acquire the lock twice, causing immediate, non-reentrant deadlocks.

The Fix & Mitigation: Consolidated connection management. Ensured the database helper closes lingering connections instantly, and updated the alert handler to reuse active transaction contexts safely without

double-locking.

Issue 4: No Hot-Reloading for Camera Configuration Changes

Severity: **CRITICAL** | **File:** backend_runner.py

The Problem: When an admin added, deleted, or modified cameras in the web UI, the backend surveillance process pool had no way to apply changes without a manual, disruptive system-wide restart.

The Fix & Mitigation: Developed an automatic filesystem watchdog thread in backend_runner.py that monitors the modification time of cameras.json. Upon detecting a change, the manager dynamically kills and respawns the relevant worker process pools in the background without affecting other streams.

Issue 5: Ephemeral Session Tokens Invalidated on Server Restart

Severity: **CRITICAL** | **File:** backend/auth.py

The Problem: Active session tokens were stored solely in server RAM. Any web-server reload, crash, or automatic process recycling forced all monitoring users (like school principals) to log in again.

The Fix & Mitigation: Created a persistent active_tokens table in the database schema. Refactored the authentication engine to write and validate session tokens from the database, retaining active dashboard logins across server restarts.

Issue 6: Non-Atomic Zone Configuration Writes

Severity: **HIGH** | **File:** zone_manager.py

The Problem: Writing new zone rules truncated the zones.json file directly. A system crash or power outage during the write window corrupted the file, destroying all camera zones.

The Fix & Mitigation: Refactored the persistence logic to perform atomic file replacement. Zone updates are now written to a temporary configuration file first, followed by an atomic os.replace() operation.

Issue 7: Concurrently Accessed Cameras File Lacked Thread Locking

Severity: **HIGH** | **File:** camera_registry.py

The Problem: Multiple processes and dashboard API requests read cameras.json concurrently while administrators wrote to it, triggering file-sharing violations and JSON parsing crashes.

The Fix & Mitigation: Introduced a dedicated reentrant thread lock (_registry_lock) protecting all disk read/write actions on the camera registry file.

Issue 8: RTSP Live Feeds Detected as Offline

Severity: HIGH | **File:** camera_registry.py

The Problem: The online check logic tested RTSP stream strings against local disk paths (using `os.path.exists`), causing all live IP cameras to display as offline/broken in the dashboard.

The Fix & Mitigation: Reconfigured the logic to accept standard network schemas (`rtsp://`, `http://`, `https://`) as valid online inputs, restoring correct status indicators.

Issue 9: Ineffective Logout Invalidation

Severity: HIGH | **File:** backend/server.py

The Problem: The logout API endpoint returned a success status but never invalidated the bearer token on the server, allowing stolen tokens to remain active.

The Fix & Mitigation: Updated the logout handler to query and remove the token record from the SQLite `active_tokens` registry, forcing server-side invalidation.

Issue 10: Insecure CORS Policy Allowing All Origins

Severity: HIGH | **File:** backend/server.py

The Problem: The web server used unrestricted CORS wildcards (*) while allowing credentials, exposing the system to Cross-Site Request Forgery (CSRF) exploits on the local network.

The Fix & Mitigation: Hardened the CORS policy to deny credentials under wildcard settings (`allow_credentials=False`), mitigating local network hijack attempts.

Issue 11: Unbounded Database Log Growth

Severity: HIGH | **File:** db_schema.py

The Problem: Surveillance tracking data wrote rows for every single person and vehicle on every processed frame, accumulating millions of rows, slowing query lookups, and filling the disk.

The Fix & Mitigation: Created a background `storage_cleanup_loop` in the server that automatically runs every hour and deletes tracking logs older than 30 days, keeping database files light and fast.

Issue 12: Dangerous Video Upload Data Loss Wipes

Severity: HIGH | **File:** backend/server.py

The Problem: Uploading a video for offline reprocessing automatically triggered `clear_event_logs()`, wiping the entire history of the 24/7 active campus cameras.

The Fix & Mitigation: Removed the global database clear commands and isolated reprocessing events, preventing accidental database wipes during manual uploads.

Issue 13: Linear Alert Snapshots Search Bottleneck

Severity: HIGH | **File:** backend/server.py

The Problem: To serve alert snapshot images, the server fetched a list of up to 1,000 alerts and scanned them one-by-one, causing massive dashboard lag under high alert counts.

The Fix & Mitigation: Replaced the linear loop with a direct, fast indexed lookup `get_alert_by_id` inside the query layer, dropping retrieval times from seconds to milliseconds.

Issue 14: No Login Rate Limiting (Brute-Force Vulnerability)

Severity: MEDIUM | **File:** backend/server.py

The Problem: The login API lacked a password attempt lock, leaving admin and principal accounts vulnerable to rapid brute-force dictionary attacks.

The Fix & Mitigation: Implemented an active lockout monitor in `auth.py`. If a user fails to authenticate 5 consecutive times, the account is locked for 5 minutes. Added an automated cleanup mechanism to prevent RAM leaks from random username spam.

Issue 15: Permanent Risk Accrual in Incident Manager

Severity: MEDIUM | **File:** incident_manager.py

The Problem: Student risk scores accumulated continuously without decaying over time, labeling everyone who regularly walked by the cameras as 'HIGH RISK' after a few days.

The Fix & Mitigation: Rewrote the risk scoring engine to implement an exponential temporal decay. Risk scores now naturally fade back to zero if no new suspicious behaviors are detected within 10 minutes. Added a thread lock to protect the profile cache.

Issue 16: Accumulating Zombie Process Handles

Severity: MEDIUM | **File:** backend_runner.py

The Problem: When child pool workers were cycled or terminated, the runner didn't call `join()`, leaving defunct zombie process handles in memory.

The Fix & Mitigation: Enforced strict process clean-ups by calling `.join(timeout=1.0)` and `.close()` on all terminated workers.

Issue 17: Broken Dwell Times for RTSP Feeds

Severity: MEDIUM | **File:** app.py

The Problem: The system used OpenCV video frame counts to compute time. For live RTSP streams, the frame count properties return 0 or -1, disabling running and fall detectors for live feeds.

The Fix & Mitigation: Added a fallback stopwatch logic that uses absolute system clock time (`time.time()`) to calculate accurate relative dwell times if stream properties fail.

Issue 18: WebSockets Set Iteration Crash (Thread-Safety)

Severity: MEDIUM | **File:** backend/server.py

The Problem: Live alert WebSocket endpoints mutated a shared set from the main event loop while worker threads iterated over it, triggering race condition crashes.

The Fix & Mitigation: Synchronized iteration using asyncio-safe thread scheduling to prevent concurrent modification errors.

Issue 19: Unbounded Image Folder Size (Disk Exhaustion)

Severity: MEDIUM | **File:** alerts.py

The Problem: Alert snapshot images were saved directly to disk. With high alert triggers, this directory could grow unbounded and fill the hard drive.

The Fix & Mitigation: Added a disk-watchdog watchdog inside `storage_cleanup_loop`. If free storage falls below 10%, it immediately purges the oldest saved alerts and snapshots until 15% is free.

Issue 20: Erratic Bounding Boxes & Ghost Detections

Severity: MEDIUM | **File:** detector.py / tracker.py

The Problem: YOLO and ByteTrack thresholds were set to 20% confidence, introducing significant background noise, false positive alerts, and fragmented tracking IDs.

The Fix & Mitigation: Fine-tuned both confidence thresholds to 45% (0.45), optimizing precision for a real school environment without losing tracking consistency.
